

Proyecto Final

K Monitor



UNIVERSIDAD
MARIANO GÁLVEZ

Catedrático

Ing. René Ornelis

Curso

Sistemas Operativos II

Integrantes

Juan José Suar Panjoj 1090-23-8522

Jhoselyn Nayelhy López Ramos 4890-20-26707

Josseline María Portillo Ramirez 0907-18-18829

6 de Junio del 2026

ÍNDICE

1.	INTRODUCCIÓN	1
2.	OBJETIVO DEL PROYECTO	2
3.	ENTORNO DE DESARROLLO	2
4.	ARQUITECTURA DE LA SOLUCIÓN.....	2
4.1	FUNCIÓN DE INICIALIZACIÓN (KMONITOR_INIT)	2
4.2	FUNCIÓN DE SALIDA (KMONITOR_EXIT).....	3
4.3	FUNCIÓN DE LECTURA (KMONITOR_SHOW).....	3
4.4	INTERFAZ /PROC.....	3
5	DIAGRAMA DE ARQUITECTURA DEL MÓDULO KMONITOR.....	4
6	FLUJO DE FUNCIONAMIENTO DEL MÓDULO	5
7	DESCRIPCIÓN DEL CONTENIDO DE CADA ARCHIVO FUENTE	5
7.1	KMONITOR.C	5
7.2	MAKEFILE.....	6
7.3	README.TXT	6
8	EXPLICACIÓN DE LAS FUNCIONES PRINCIPALES	6
8.1	GET_STATE().....	6
8.2	KMONITOR_SHOW().....	7
8.3	KMONITOR_OPEN()	7
8.4	KMONITOR_INIT()	7
8.5	KMONITOR_EXIT()	7
9.	EXPLICACIÓN DE LAS ESTRUCTURAS DE DATOS DEL KERNEL UTILIZADAS.....	8
9.1	STRUCT SYSINFO	8
9.2	STRUCT TASK_STRUCT	8
9.3	FOR_EACH_PROCESS	8
9.4	/PROC (PROCFS).....	9
9.5	SEQ_FILE	9
9.6	PROC_OPS.....	9
9.7	PROC_DIR_ENTRY	10
10.	COMANDOS UTILIZADOS DURANTE LAS PRUEBAS.....	10
11.	EVIDENCIAS DE FUNCIONAMIENTO.....	11
11.1	COMPILACIÓN EXITOSA DEL MÓDULO DEL KERNEL	11
11.2	CARGA DEL MÓDULO Y CREACIÓN DE LA ENTRADA EN /PROC	11
11.3	LECTURA DE INFORMACIÓN DE MEMORIA Y PROCESOS	11
11.4	VERIFICACIÓN DE MENSAJES DEL KERNEL	15
11.5	ELIMINACIÓN DEL MÓDULO Y LIMPIEZA DE LA ENTRADA EN /PROC.....	15
12.	CONCLUSIONES.....	16

1. Introducción

El sistema operativo Linux se caracteriza por ofrecer una arquitectura robusta y flexible que permite interactuar directamente con los componentes internos del kernel mediante módulos cargables. Estos módulos facilitan la incorporación de nuevas funcionalidades sin necesidad de recompilar el núcleo completo, proporcionando una forma eficiente y dinámica de extender las capacidades del sistema operativo.

En el presente proyecto se desarrolló **KMonitor**, un Módulo Cargable del Kernel (LKM) diseñado para obtener y mostrar información interna del sistema operativo Linux a través del sistema de archivos virtual **/proc**. Mediante este mecanismo es posible consultar información relacionada con la memoria del sistema y los procesos registrados en el kernel, incluyendo datos como el identificador del proceso (PID), nombre y estado de ejecución.

El desarrollo de KMonitor permitió aplicar conocimientos relacionados con la administración de procesos, manejo de memoria, estructuras internas del kernel y comunicación entre el espacio de kernel y el espacio de usuario. Para ello se utilizaron mecanismos y estructuras propias de Linux como **struct sysinfo**, **task_struct**, **for_each_process**, **procfs** y **seq_file**, los cuales desempeñan un papel fundamental en la obtención y presentación de información del sistema.

De esta manera, el proyecto constituye una aplicación práctica de los conceptos estudiados sobre la arquitectura interna de Linux, demostrando cómo un módulo del kernel puede integrarse dinámicamente para recolectar métricas del sistema y ponerlas a disposición del usuario en tiempo real mediante una interfaz sencilla y accesible.

2. Objetivo del proyecto

El proyecto KMonitor tiene como propósito desarrollar un Módulo Cargable del Kernel (LKM) para Linux capaz de obtener información de memoria y procesos del sistema, exponiéndola al usuario mediante el sistema de archivos virtual /proc.

A través de este módulo es posible visualizar la memoria total, memoria libre, porcentaje de uso y un listado de procesos con su PID, nombre y estado, permitiendo aplicar conceptos relacionados con la administración de procesos, manejo de memoria y comunicación entre el espacio de kernel y el espacio de usuario.

3. Entorno de desarrollo

El proyecto fue desarrollado y probado en un entorno Linux utilizando una máquina virtual para evitar riesgos sobre el sistema operativo anfitrión.

1.1 Características del entorno utilizado:

- Sistema operativo Linux Ubuntu.
- Uso de terminal Linux para compilación y ejecución.
- Headers del kernel instalados para compilación de módulos.
- Uso de permisos administrativos mediante sudo.
- Compilación utilizando Makefile.

4. Arquitectura de la solución

El módulo KMonitor está diseñado como un Módulo Cargable del Kernel (LKM) que se ejecuta en el espacio del kernel de Linux, permitiendo integrarse dinámicamente al sistema operativo para obtener y mostrar información interna sin modificar el núcleo de forma permanente.

La arquitectura del sistema se compone de los siguientes elementos:

4.1 Función de inicialización (kmonitor_init)

Esta función se ejecuta automáticamente al cargar el módulo mediante el comando:

➤ **sudo insmod kmonitor.ko**

Sus responsabilidades son:

- crear la entrada `/proc/kmonitor_grupo6`,
- definir permisos de solo lectura (0444),
- verificar que la creación sea exitosa,
- registrar mensajes informativos en el kernel mediante `pr_info()`.

4.2 Función de salida (kmonitor_exit)

Esta función se ejecuta automáticamente al remover el módulo mediante el comando:

➤ **`sudo rmmod kmonitor`**

Sus responsabilidades son:

- eliminar la entrada creada en `/proc`,
- liberar los recursos utilizados por el módulo,
- registrar mensajes informativos en el kernel mediante `pr_info()`.

4.3 Función de lectura (kmonitor_show)

Es la función principal encargada de obtener y presentar la información generada por el módulo.

Sus responsabilidades son:

- obtener información de memoria del sistema,
- calcular el porcentaje de uso de memoria,
- recorrer la lista de procesos registrados en el sistema
- mostrar PID, nombre y estado de los procesos,
- formatear la salida mediante `seq_file`.

4.4 Interfaz /proc

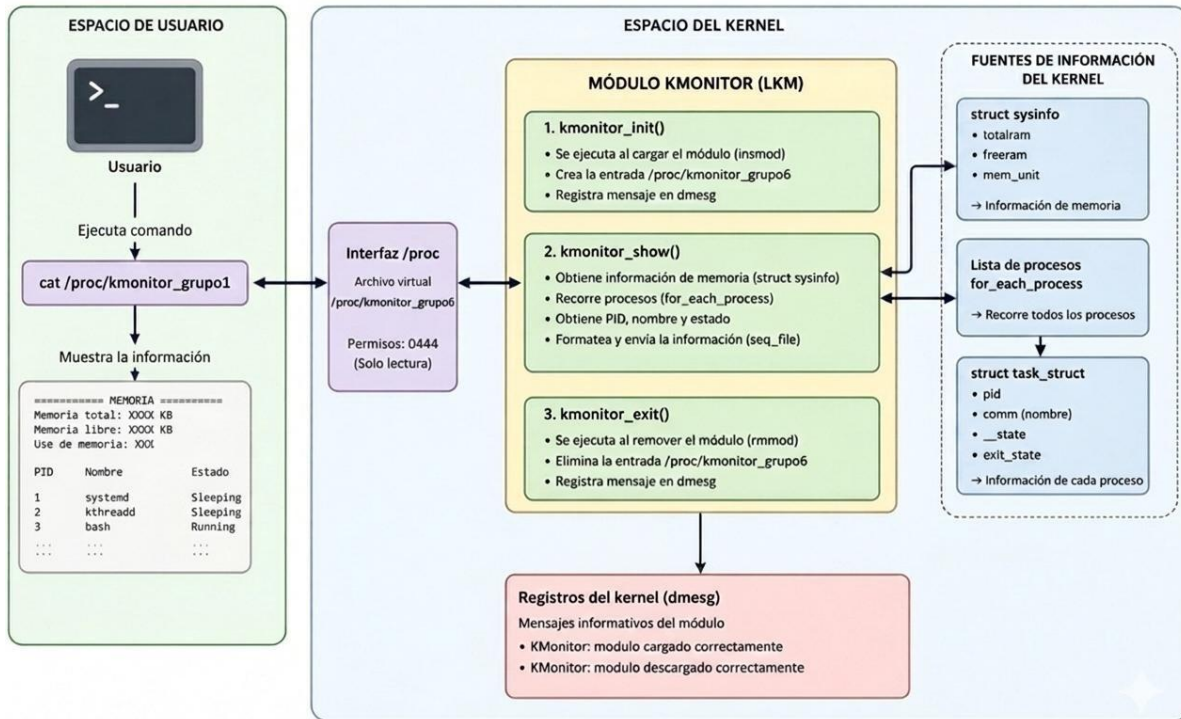
El módulo utiliza el sistema de archivos virtual **/proc** como mecanismo de comunicación entre el espacio del kernel y el espacio de usuario, permitiendo consultar la información generada por KMonitor de forma sencilla desde la terminal.

La información puede visualizarse mediante el comando:

➤ **`cat /proc/kmonitor_grupo6`**

Al ejecutar este comando, el kernel invoca la función `kmonitor_show()`, la cual genera y presenta la información de memoria y procesos recopilada por el módulo.

5 Diagrama de arquitectura del módulo Kmonitor



Explicación del diagrama

El usuario interactúa con el módulo mediante la lectura del archivo `/proc/kmonitor_grupo6`; al consultar este archivo, el kernel ejecuta la función `kmonitor_show()`, la cual obtiene información de memoria utilizando `struct sysinfo` y recorre la lista de procesos mediante `for_each_process` y `task_struct`.

La información recopilada es organizada con `seq_file` y posteriormente enviada al espacio de usuario para ser visualizada en la terminal, permitiendo consultar el estado del sistema de forma sencilla y en tiempo real.

6 Flujo de funcionamiento del módulo

El funcionamiento de KMonitor inicia cuando el usuario compila el código fuente y carga el archivo `kmonitor.ko` en el kernel mediante `insmod`, momento en el que se ejecuta automáticamente la función `kmonitor_init()`, encargada de crear la entrada virtual `/proc/kmonitor_grupo6`.

Cuando el usuario ejecuta:

➤ **`cat /proc/kmonitor_grupo6`**

el kernel invoca la función `kmonitor_show()`, la cual obtiene información de memoria mediante `struct sysinfo` y recorre la lista de procesos del sistema utilizando `for_each_process` y `task_struct`.

La información recopilada se organiza mediante `seq_file`, mostrando:

- memoria total,
- memoria libre,
- porcentaje de uso de memoria,
- PID,
- nombre del proceso,
- estado del proceso.

Al finalizar las pruebas, el usuario puede descargar el módulo mediante:

➤ **`sudo rmmod kmonitor`**

lo que provoca la ejecución de `kmonitor_exit()`, función encargada de eliminar la entrada creada en `/proc` y liberar los recursos asociados al módulo.

7 Descripción del contenido de cada archivo fuente

El proyecto está compuesto por los siguientes archivos:

7.1 `kmonitor.c`

Archivo principal que contiene la implementación del módulo del kernel.

Incluye:

- funciones de inicialización y salida,
- función de lectura `kmonitor_show`,

- función auxiliar `get_state`,
- manejo de memoria y procesos,
- creación y eliminación de la entrada en `/proc`,
- configuración de operaciones mediante `proc_ops`.

7.2 Makefile

Archivo encargado de compilar el módulo del kernel.

Define:

- el nombre del módulo a compilar (`kmonitor.o`)
- la regla `all` para generar el archivo `.ko`,
- la regla `clean` para eliminar archivos generados durante la compilación.

7.3 README.txt

Archivo de texto que describe el funcionamiento básico del proyecto: cómo compilar, ejecutar y probar el módulo.

Incluye:

- instrucciones para compilar el módulo,
- comandos para cargar y descargar el módulo,
- forma de consultar la información con `cat`,
- descripción general del proyecto

8 Explicación de las funciones principales

8.1 `get_state()`

Función auxiliar encargada de convertir el estado interno de un proceso en un texto legible para el usuario.

Estados utilizados:

- Running
- Sleeping
- Zombie

Para ello, la función analiza los campos internos del proceso y determina el estado que será mostrado al usuario.

Utiliza:

- **task->exit_state** para detectar procesos zombie,
- **task->__state** para identificar si el proceso se encuentra ejecutándose.

8. 2 kmonitor_show()

Función principal encargada de generar la información mostrada en /proc/kmonitor_grupo6.

Responsabilidades:

- obtener información de memoria,
- calcular el porcentaje de uso de memoria,
- recorrer todos los procesos registrados en el sistema,
- mostrar la tabla con PID, nombre y estado.

8.3 kmonitor_open()

Función ejecutada cuando el usuario abre el archivo ubicado en /proc, se utiliza junto con single_open() para conectar la lectura del archivo con la función kmonitor_show().

8.4 kmonitor_init()

Función de inicialización del módulo, se ejecuta al cargar el módulo mediante insmod.

Responsabilidades:

- crear la entrada /proc/kmonitor_grupo6,
- verificar errores durante su creación,
- mostrar mensajes informativos mediante pr_info().

8.5 kmonitor_exit()

Función de limpieza del módulo, se ejecuta al remover el módulo con rmmod.

Responsabilidades:

- eliminar la entrada creada en /proc,
- liberar recursos asociados,
- mostrar mensajes informativos en dmesg

9. Explicación de las estructuras de datos del kernel utilizadas

El módulo utiliza diferentes estructuras y mecanismos internos del kernel Linux para obtener información del sistema.

9.1 struct sysinfo

Estructura utilizada para obtener información general de memoria del sistema.

Campos utilizados:

- `totalram` → Memoria total disponible
- `freeram` → Memoria libre
- `mem_unit` → Unidad de medida utilizada por el kernel

Con estos datos se obtiene:

- Memoria total en KB
- Memoria libre en KB

Posteriormente, la función `kmonitor_show()` utiliza esta información para calcular el porcentaje de uso de memoria.

9.2 struct task_struct

Es la estructura principal que representa un proceso dentro del kernel de Linux.

Campos utilizados:

- `pid` → Identificador del proceso
- `comm` → Nombre del proceso
- `__state` → Estado del proceso
- `exit_state` → usado para identificar procesos zombie.

9.3 for_each_process

Es una macro del kernel que permite iterar sobre todos los procesos activos del sistema.

Se utiliza para recorrer todos los procesos del sistema de forma segura sin manipular manualmente las estructuras enlazadas del kernel

.Ejemplo de uso en el módulo:

```
for_each_process(task) {  
    seq_printf(m, "%-8d %-25s %-  
        12s\n", task->pid,  
        task->comm,  
        get_state(task))  
}
```

9.4 /proc (procfs)

Sistema de archivos virtual administrado por el kernel Linux y a diferencia de un sistema de archivos tradicional, la información mostrada en /proc es generada dinámicamente por el kernel.

En este proyecto se utiliza para:

- crear el archivo /proc/kmonitor_grupo6,
- mostrar la información del sistema,
- permitir la consulta desde el espacio de usuario mediante cat.

9.5 seq_file

Interfaz del kernel usada para mostrar información de forma ordenada y segura dentro de archivos /proc y es adecuada para salidas de varias líneas, como listados de procesos.

9.6 proc_ops

Estructura utilizada para definir las operaciones asociadas al archivo creado en /proc.

Incluye:

- apertura del archivo,
- lectura,
- desplazamiento,
- liberación de recursos.

Esta estructura representa la forma moderna de manejar archivos /proc en kernels Linux actuales.

9.7 proc_dir_entry

Estructura utilizada para almacenar la referencia de la entrada, creada en /proc. En el proyecto se utiliza mediante:

- **static struct proc_dir_entry**

***kmonitor_entry; Permite:**

- verificar si la entrada fue creada correctamente,
- eliminar la entrada utilizando proc_remove() durante la limpieza del módulo.

10. Comandos utilizados durante las pruebas

Comando	Descripción
make	Compila el módulo del kernel utilizando el archivo Makefile y genera el archivo ejecutable kmonitor.ko.
sudo insmod kmonitor.ko	Carga el módulo kmonitor.ko dentro del kernel Linux y ejecuta la función de inicialización del módulo.
lsmod grep kmonitor	Muestra la lista de módulos actualmente cargados en el kernel y verifica si el módulo kmonitor se encuentra activo.
ls /proc grep kmonitor	Busca dentro del sistema de archivos /proc la entrada creada por el módulo para confirmar que fue generada correctamente.
cat /proc/kmonitor_grupo6	Lee y muestra la información generada por el módulo, incluyendo memoria del sistema y lista de procesos activos.
sudo dmesg tail	Muestra los mensajes más recientes del kernel Linux, permitiendo verificar mensajes relacionados con la carga y descarga del módulo.
sudo rmmod kmonitor	Remueve el módulo kmonitor del kernel y ejecuta la función de limpieza del módulo.
ls /proc grep kmonitor	Verifica que la entrada creada en /proc haya sido eliminada correctamente después de descargar el módulo.

11. Evidencias de funcionamiento

Durante las pruebas realizadas se verificó correctamente lo siguiente:

11.1 Compilación exitosa del módulo del kernel

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~$ cd kmonitor
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ sudo rmmod kmonitor 2>/dev/null
[sudo] contraseña para jhoselyn-l-pe:
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ make
make -C /lib/modules/6.17.0-22-generic/build M=/home/jhoselyn-l-pe/kmonitor modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.17.0-22-generic'
make[2]: se entra en el directorio '/home/jhoselyn-l-pe/kmonitor'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] kmonitor.o
MODPOST Module.symvers
CC [M] kmonitor.mod.o
CC [M] .module-common.o
LD [M] kmonitor.ko
BTF [M] kmonitor.ko
Skipping BTF generation for kmonitor.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/jhoselyn-l-pe/kmonitor'
make[1]: se sale del directorio '/usr/src/linux-headers-6.17.0-22-generic'
```

Se muestra la compilación correcta del módulo kmonitor.ko utilizando el comando make, generando exitosamente el archivo ejecutable del módulo del kernel Linux.

11.2 Carga del módulo y creación de la entrada en /proc

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ sudo insmod kmonitor.ko
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ lsmod | grep kmonitor
kmonitor                12288  0
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ ls /proc | grep kmonitor
kmonitor_grupo6
```

Se verifica la carga correcta del módulo mediante sudo insmod kmonitor.ko y la creación de la entrada virtual /proc/kmonitor_grupo6.

11.3 Lectura de Información de Memoria y Procesos

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ cat /proc/kmonitor_grupo6
===== MEMORIA =====
Memoria total: 4008728 KB
Memoria libre: 211112 KB
Uso de memoria: 94%

PID      Nombre                               Estado
1        systemd                             Sleeping
2        kthreadd                             Sleeping
3        pool_workqueue_                      Sleeping
4        kworker/R-rcu_g                      Sleeping
5        kworker/R-sync_                      Sleeping
6        kworker/R-kvfre                      Sleeping
7        kworker/R-slub_                      Sleeping
8        kworker/R-netns                      Sleeping
12       kworker/u8:0                         Sleeping
13       kworker/R-mm_pe                      Sleeping
14       ksoftirqd/0                          Sleeping
15       rcu_preempt                          Sleeping
16       rcu_exp_par_gp_                      Sleeping
17       rcu_exp_gp_kthr                      Sleeping
18       migration/0                          Sleeping
```

Se muestra el inicio de la lectura del archivo `/proc/kmonitor_grupo6`, donde se visualiza la información de memoria total, memoria libre y porcentaje de uso del sistema.

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor
24      ksoftirqd/1      Sleeping
29      kdevtmpfs      Sleeping
30      kworker/R-inet_ Sleeping
31      rcu_tasks_kthre Sleeping
32      rcu_tasks_rude_ Sleeping
33      rcu_tasks_trace Sleeping
34      kauditd         Sleeping
35      khungtaskd      Sleeping
36      oom_reaper      Sleeping
39      kworker/R-write Sleeping
40      kcompactd0      Sleeping
41      ksmd            Sleeping
42      khugepaged      Sleeping
43      kworker/R-kbloc Sleeping
44      kworker/R-blkcg  Sleeping
45      kworker/R-kinte  Sleeping
46      irq/9-acpi      Sleeping
47      kworker/R-tpm_d  Sleeping
48      kworker/R-ata_s  Sleeping
49      kworker/R-md     Sleeping
50      kworker/R-md_bi  Sleeping
51      kworker/R-edac-  Sleeping
52      kworker/R-devfr  Sleeping
53      watchdogd       Sleeping
55      kworker/R-quota  Sleeping
58      kswapd0         Sleeping
```

Continuación del listado de procesos obtenidos desde el kernel Linux mediante la macro `for_each_process`.

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor
60      kworker/R-kthro  Sleeping
61      kworker/R-acpi_  Sleeping
64      scsi_eh_0        Sleeping
65      kworker/R-scsi_  Sleeping
66      scsi_eh_1        Sleeping
67      kworker/R-scsi_  Sleeping
70      kworker/R-mlb    Sleeping
71      kworker/R-ipv6_  Sleeping
72      kworker/u8:1     Sleeping
73      kworker/R-kstrp  Sleeping
75      kworker/u11:0    Sleeping
76      kworker/u12:0    Sleeping
77      kworker/u13:0    Sleeping
89      kworker/R-charg  Sleeping
140     scsi_eh_2        Sleeping
141     kworker/R-scsi_  Sleeping
153     kworker/R-ttm    Sleeping
218     jbd2/sda2-8      Sleeping
219     kworker/R-ext4-  Sleeping
267     systemd-journal Sleeping
340     systemd-udev    Sleeping
385     psimon          Sleeping
395     systemd-oomd     Sleeping
398     systemd-resolve  Sleeping
595     avahi-daemon     Sleeping
596     dbus-daemon      Sleeping
```

Se observa parte de los procesos internos del sistema operativo y sus respectivos estados.

jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor		
621	polkitd	Sleeping
622	power-profiles-	Sleeping
634	snapp	Sleeping
644	accounts-daemon	Sleeping
652	cron	Sleeping
656	switcheroo-cont	Sleeping
674	systemd-logind	Sleeping
676	udisksd	Sleeping
736	avahi-daemon	Sleeping
767	NetworkManager	Sleeping
774	wpa_supplicant	Sleeping
789	rsyslogd	Sleeping
847	ModemManager	Sleeping
1146	unattended-upgr	Sleeping
1171	kerneloops	Sleeping
1178	kerneloops	Sleeping
1984	VBoxService	Sleeping
2465	gdm3	Sleeping
2514	rtkit-daemon	Sleeping
2609	colord	Sleeping
2648	upowerd	Sleeping
2968	gdm-session-wor	Sleeping
2980	systemd	Sleeping
2985	(sd-pam)	Sleeping
2996	pipewire	Sleeping
2997	pipewire	Sleeping

Evidencia adicional del monitoreo de procesos realizado por el módulo KMonitor dentro del sistema Linux

jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor		
3001	pipewire-pulse	Sleeping
3008	gnome-keyring-d	Sleeping
3017	dbus-daemon	Sleeping
3053	gdm-wayland-ses	Sleeping
3086	gnome-session-b	Sleeping
3087	xdg-document-po	Sleeping
3107	xdg-permission-	Sleeping
3119	fusermount3	Sleeping
3156	gcr-ssh-agent	Sleeping
3157	gnome-session-c	Sleeping
3164	gvfsd	Sleeping
3184	gvfsd-fuse	Sleeping
3188	gnome-session-b	Sleeping
3228	gnome-shell	Sleeping
3230	at-spi-bus-laun	Sleeping
3246	dbus-daemon	Sleeping
3292	at-spi2-registr	Sleeping
3312	gnome-shell-cal	Sleeping
3314	dconf-service	Sleeping
3324	evolution-sourc	Sleeping
3333	gjs	Sleeping
3334	Xwayland	Sleeping
3345	ibus-daemon	Running
3346	gsd-a11y-settin	Sleeping
3350	gsd-color	Sleeping
3351	gsd-datetime	Sleeping

Se muestran procesos relacionados con el entorno gráfico y servicios del sistema Linux.

jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor		
3351	gsd-datetime	Sleeping
3354	gsd-housekeepin	Sleeping
3355	gsd-keyboard	Sleeping
3357	gsd-media-keys	Sleeping
3358	gsd-power	Sleeping
3361	gsd-print-notif	Sleeping
3362	gsd-rfkill	Sleeping
3363	gsd-screensaver	Sleeping
3365	gsd-sharing	Sleeping
3371	gsd-smartcard	Sleeping
3373	gsd-sound	Sleeping
3374	gsd-wacom	Sleeping
3377	snapped-desktop-i	Sleeping
3381	evolution-alarm	Sleeping
3392	gsd-disk-utilit	Sleeping
3500	gsd-printer	Sleeping
3591	gvfs-udisks2-vo	Sleeping
3594	ibus-dconf	Sleeping
3597	goa-daemon	Sleeping
3598	snapped-desktop-i	Sleeping
3603	ibus-extension-	Sleeping
3605	ibus-portal	Sleeping
3636	goa-identity-se	Sleeping
3637	gvfs-gphoto2-vo	Sleeping
3653	evolution-calen	Sleeping
3659	gvfs-mtp-volume	Sleeping

Continuación de la lista de procesos administrados por el kernel Linux.

jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor		
3671	gvfs-afc-volume	Sleeping
3686	evolution-addre	Sleeping
3711	ibus-engine-sim	Sleeping
3748	gvfsd-metadata	Sleeping
3749	gvfsd-trash	Sleeping
3818	gjs	Sleeping
3848	gsd-xsettings	Sleeping
3875	xdg-desktop-por	Sleeping
3892	ibus-x11	Sleeping
3894	tracker-miner-f	Sleeping
3895	xdg-desktop-por	Sleeping
3904	mutter-x11-fram	Sleeping
3906	VBoxClient	Sleeping
3908	VBoxClient	Sleeping
3948	VBoxClient	Sleeping
3949	VBoxClient	Sleeping
3981	xdg-desktop-por	Sleeping
4084	update-notifier	Sleeping
4768	fwupd	Sleeping
5875	cupsd	Sleeping
5876	cups-browsed	Sleeping
6641	kworker/u10:0	Sleeping
6648	kworker/u9:3	Sleeping
6996	kworker/0:0H	Sleeping
11635	kworker/u10:1	Sleeping
17896	psimon	Sleeping

Evidencia adicional del monitoreo de procesos ejecutándose dentro del sistema operativo.


```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox: ~/kmonitor
11635 kworker/u10:1 Sleeping
17896 psimon Sleeping
22016 kworker/0:1H Sleeping
22903 kworker/0:3 Sleeping
22918 kworker/u9:2 Sleeping
22919 kworker/0:0 Sleeping
22921 kworker/1:0H Sleeping
23036 gjs Sleeping
23088 kworker/u10:3 Sleeping
23206 kworker/u10:4 Sleeping
23456 kworker/1:2 Sleeping
23512 gnome-terminal Sleeping
23515 gnome-terminal Sleeping
23521 gnome-terminal Sleeping
23529 bash Sleeping
23811 kworker/1:2H Sleeping
23829 kworker/1:1 Sleeping
23830 kworker/u9:1 Sleeping
23838 nautilus Sleeping
23882 gvfsd-recent Sleeping
23908 eog Sleeping
23918 kworker/1:1H Sleeping
23922 kworker/0:1 Sleeping
23975 kworker/u10:2 Sleeping
23977 cat Running
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$
```

Se muestra la parte final de la salida generada por el módulo, incluyendo procesos del sistema y el comando cat utilizado para consultar la información.

11.4 Verificación de mensajes del kernel

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ sudo dmesg | tail
[18499.835252] workqueue: vmstat_update hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[18792.785165] workqueue: blk_mq_requeue_work hogged CPU for >10000us 131 times, consider switching to WQ_UNBOUND
[19301.592591] workqueue: inode_switch_wbs_work_fn hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[19301.616126] workqueue: inode_switch_wbs_work_fn hogged CPU for >10000us 5 times, consider switching to WQ_UNBOUND
[19301.640119] workqueue: inode_switch_wbs_work_fn hogged CPU for >10000us 7 times, consider switching to WQ_UNBOUND
[19301.730225] workqueue: inode_switch_wbs_work_fn hogged CPU for >10000us 11 times, consider switching to WQ_UNBOUND
[19301.923967] workqueue: inode_switch_wbs_work_fn hogged CPU for >10000us 19 times, consider switching to WQ_UNBOUND
[19303.871394] workqueue: inode_switch_wbs_work_fn hogged CPU for >10000us 35 times, consider switching to WQ_UNBOUND
[21046.195388] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 515 times, consider switching to WQ_UNBOUND
[22518.409121] KMonitor: modulo cargado correctamente
```

Se verifican los mensajes generados por el módulo utilizando el comando `sudo dmesg | tail`, confirmando la carga correcta del módulo dentro del kernel Linux.

11.5 Eliminación del módulo y limpieza de la entrada en /proc

```
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ sudo rmmod kmonitor
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$ ls /proc | grep kmonitor
jhoselyn-l-pe@jhoselyn-l-pe-VirtualBox:~/kmonitor$
```

Se muestra la eliminación correcta del módulo mediante `sudo rmmod kmonitor` y la verificación de que la entrada `/proc/kmonitor_grupo6` fue eliminada exitosamente.

12. Conclusiones

El desarrollo de KMonitor permitió comprender de forma práctica el funcionamiento interno del kernel Linux, demostrando cómo un módulo cargable puede integrarse dinámicamente al sistema operativo para obtener y exponer información del sistema mediante el sistema de archivos virtual /proc.

A través de la implementación del módulo fue posible obtener información de memoria y mostrar el estado de los procesos registrados en el sistema, aplicando conceptos relacionados con la administración de recursos, la gestión de procesos y la comunicación entre el espacio de kernel y el espacio de usuario.

Asimismo, el proyecto facilitó la aplicación de estructuras y mecanismos fundamentales de Linux como struct sysinfo, task_struct, proc_ops, seq_file y la macro for_each_process, fortaleciendo la comprensión de la arquitectura interna del sistema operativo y de las técnicas utilizadas para el monitoreo de información en tiempo real.

Las pruebas realizadas confirmaron la correcta compilación, carga, ejecución y eliminación del módulo, evidenciando el adecuado funcionamiento de la solución desarrollada y consolidando conocimientos prácticos sobre programación a nivel kernel, administración de recursos y monitoreo básico de sistemas Linux.